

ДЗ №12 по ОС.

Магомедов Абдул Омаргаджиевич

БПИ 234

Условие:

До 8 баллов

Реализовать для родительского и дочернего процесса двухстороннюю связь с использованием только одного неименованного канала (*pipe*) и циклической организацией обмена посредством семафора *Unix System V*. При этом канал должен быть постоянно открытым. Интервал для обмена должен быть равен одной секунде. Обмен должен завершаться после поочередной передачи в каждую сторону по десять сообщений. По завершении обмена обеспечить корректное закрытие канала и удаление семафора.

Опционально +2 балла

Вместо десяти обменов организовать поочередный обмен, завершающийся посылкой сигнала прерывания с клавиатуры родительскому или дочернему процессу. При этом оба процесса должны корректно завершить работу с закрытием канала и удалением семафора.

Предоставить один вариант программы только на планируемую оценку.

Цель программы:

Программа реализует двустороннюю связь между родительским и дочерним процессами с использованием одного неименованного канала (pipe) и семафора System V. Обмен сообщениями осуществляется поочерёдно с интервалом в 1 секунду. Программа завершается корректно после получения сигнала прерывания с клавиатуры (Ctrl+C).

Как это реализовано

1. Создание канала (pipe)

Вызов функции `pipe(fd)` создаёт два файловых дескриптора — для чтения и записи.

Один канал используется для передачи сообщений между родительским и дочерним процессами в обе стороны.

2. Создание семафора

С помощью `semget` инициализируется набор семафоров с одним элементом.

Изначально семафор установлен в 0 (`semctl`), чтобы дочерний процесс сразу мог читать.

3. Семафор для синхронизации

- Родитель и потомок используют один семафор, поочерёдно изменяя его значение, чтобы синхронизировать доступ к каналу.
- Функция `sem_do` инкапсулирует операции `semop`, ожидая возможности для обмена.
- Родитель инициирует отправку данных, потомок читает и отвечает.

4. Процесс-ребёнок

- Читает сообщение от родителя из канала.

- Выводит его на экран.
- Отправляет ответ родителю через тот же канал.
- Использует `sem_do` для передачи «эстафеты» родителю.

5. Процесс-родитель

- Периодически (с интервалом 1 секунда) пишет сообщение в канал.
- Читает ответ от ребёнка.
- После каждого обмена «отдаёт эстафету» потомку с помощью `sem_do`.
- После получения сигнала прерывания (SIGINT) завершает цикл, уведомляет потомка, корректно закрывает канал и удаляет семафор.

6. Обработка сигнала завершения

- При нажатии Ctrl+C (SIGINT), устанавливается флаг завершения `terminate`.
- Процессы корректно завершаются: закрываются каналы и удаляется семафор (`semctl` с флагом `IPC_RMID`).
- На экран выводится сообщение: «Получен сигнал завершения с клавиатуры, программа завершает свою работу»

Пример работы программы:

```
arbuz@AsusVivobook:~/HW12$ ./mark10
Дочерний процесс получил: Сообщение 0 от родителя
Родительский процесс получил: Ответ 0 от дочернего процесса
Дочерний процесс получил: Сообщение 1 от родителя
Родительский процесс получил: Ответ 1 от дочернего процесса
Дочерний процесс получил: Сообщение 2 от родителя
Родительский процесс получил: Ответ 2 от дочернего процесса
Дочерний процесс получил: Сообщение 3 от родителя
Родительский процесс получил: Ответ 3 от дочернего процесса
Дочерний процесс получил: Сообщение 4 от родителя
Родительский процесс получил: Ответ 4 от дочернего процесса
^CРодительский процесс получил: Сообщение 5 от родителя
Получен сигнал завершения с клавиатуры, программа завершает свою работу
```

Т.к. программа реализована на оценку 10, то обмены будут происходить, пока не будет получен сигнал завершения.

Вывод:

Программа успешно демонстрирует синхронный обмен данными между родителем и потомком с использованием одного канала и одного семафора. Завершение работы корректно обрабатывается при получении сигнала от пользователя, как это требуется по условию задания.